



**INSTITUTO FEDERAL DE ALAGOAS
CAMPUS PALMEIRA DOS ÍNDIOS
CURSO TÉCNICO INTEGRADO AO ENSINO MÉDIO EM INFORMÁTICA**

**ALYSSON MATHEUS DE SOUZA RODRIGUES
JORDANA DA SILVA FERRO
VITOR VINÍCIUS PORANGABA TORRES**

**PESQUISA SOBRE VALIDAÇÃO
DE DADOS EM SISTEMAS WEB**

**PALMEIRA DOS ÍNDIOS-AL
2026**

ALYSSON MATHEUS DE SOUZA RODRIGUES
JORDANA DA SILVA FERRO
VITOR VINÍCIUS PORANGABA TORRES

PESQUISA SOBRE VALIDAÇÃO
DE DADOS EM SISTEMAS WEB

Trabalho elaborado na disciplina de
Programação Web para obtenção de nota.
Professor: Carlos Jean.

PALMEIRA DOS ÍNDIOS-AL
2026

Sumário:

Sumário:	3
1 Introdução	4
2 Validação Client-Side vs. Server-Side	4
2.1 Validação no lado do cliente (Client-Side).....	4
2.2 Validação no lado do servidor (Server-Side).....	5
3 Atributos de Validação Nativa do HTML5	6
3.1 Atributo required.....	6
3.2 Atributo pattern (Expressões Regulares).....	6
3.3 Seletores de tamanho minlength e maxlength.....	6
4 Estados Visuais (CSS) e Pseudo-classes :valid and :invalid	7
4.1 Pseudo-classes :valid and :invalid.....	7
4.2 Estratégias Avançadas de Usabilidade e Estilização.....	7
5 Conclusão	8
Referências	8

1 Introdução

No ecossistema do desenvolvimento de aplicações web modernas, a validação de dados constitui um pilar fundamental e indispensável. Ela impacta diretamente a segurança cibernética, a integridade dos bancos de dados relacionais e não relacionais, bem como a qualidade da Experiência do Usuário (UX). Segundo Silva (2014), entende-se por validação o conjunto estruturado de processos e regras lógicas aplicadas sobre as entradas de dados fornecidas pelos usuários para garantir que estas estejam em estrita conformidade com as diretrizes lógicas, estruturais e de negócio da aplicação antes de sua consolidação e armazenamento definitivo.

2 Validação Client-Side vs. Server-Side

A arquitetura web profissional exige que a validação ocorra de forma distribuída e redundante em duas camadas tecnológicas distintas: a camada do cliente (Client-Side) e a camada do servidor (Server-Side). Conforme apontado por Flanagan (2012), cada uma dessas vertentes possui propósitos, capacidades computacionais e níveis de confiança diametralmente opostos.

2.1 Validação no lado do cliente (Client-Side)

A validação executada no lado do cliente processa-se localmente no dispositivo e navegador web do usuário final. Ela utiliza-se tanto de recursos semânticos nativos do HTML5 quanto de scripts lógicos escritos em linguagem JavaScript. De acordo com as diretrizes do World Wide Web Consortium (W3C, 2021), a validação voltada ao cliente possui o objetivo primordial de otimizar a interface humana através de retornos imediatos.

As principais vantagens interativas deste modelo consistem na eliminação completa da latência de rede. O visitante é alertado em tempo real sobre erros corriqueiros de digitação — como a ausência do caractere arroba em endereços de correio eletrônico ou omissão de campos obrigatórios —, evitando o recarregamento desnecessário da página web (Page Refresh). Adicionalmente, atua filtrando requisições malformadas, o que preserva a largura de banda e reduz o consumo de processamento desnecessário na infraestrutura do servidor.

Todavia, sob a ótica da segurança da informação, a validação no cliente apresenta uma vulnerabilidade crítica: sua segurança é considerada nula (FLANAGAN, 2012). Dado que o código-fonte é interpretado diretamente no ecossistema do cliente, ele fica exposto à manipulação total por usuários mal-intencionados. Através das ferramentas internas de inspeção do navegador (Developer Tools), qualquer indivíduo possui a capacidade de desativar o interpretador JavaScript, remover atributos de restrição do documento HTML ou simular o envio de requisições maliciosas e corrompidas diretamente para as rotas da aplicação por meio de utilitários como cURL ou Postman.

2.2 Validação no lado do servidor (Server-Side)

Em contrapartida, a validação no lado do servidor atua no ecossistema interno e restrito da retaguarda de sistemas (Back-end), operando em ambientes isolados sob linguagens de programação estruturadas (como PHP, Node.js, Python ou Java). Conforme preconiza Silva (2014), esta camada recebe o pacote de dados encapsulado em requisições de rede (como o método GET ou POST), inspecionando rigorosamente cada parâmetro antes de autorizar transações de gravação.

A validação server-side constitui a barreira de segurança definitiva e infalível de um sistema computacional. Ela protege a aplicação contra ataques severos de injeção de código (SQL Injection), Cross-Site Scripting (XSS) e dados corrompidos que visam desestabilizar o banco de dados. Ademais, permite a centralização e o cumprimento estrito das regras de negócio complexas que dependem de consultas externas e assíncronas, como a validação de unicidade de cadastros de CPFs ou e-mails pré-existentes. A principal desvantagem reside na usabilidade de forma isolada, dado que a resposta aos erros de digitação introduz latência devido à jornada dos dados na rede.

Critério de Comparação	Validação Client-Side	Validação Server-Side
Ambiente de Execução	Navegador do Usuário (Frontend)	Servidor de Hospedagem (Backend)
Foco Principal	Usabilidade e Experiência do Usuário (UX)	Segurança da Informação e Integridade
Velocidade de Feedback	Instantânea (Síncrona)	Dependente da latência da rede (Assíncrona)
Segurança contra Fraudes	Baixa/Nula (Pode ser ignorada)	Alta/Absoluta (Inviolável pelo cliente)

3 Atributos de Validação Nativa do HTML5

A evolução técnica das especificações do HTML5 mitigou expressivamente a dependência de scripts extensos e complexos de JavaScript por intermédio da consolidação da Constraint Validation API. Conforme documentado pela Mozilla Developer Network (MDN WEB DOCS, 2023), as tags de formulário ganharam inteligência nativa por meio de atributos semânticos especializados.

3.1 Atributo required

O atributo booleano required estipula que um campo de entrada de dados torna-se estritamente obrigatório. Se o usuário tentar submeter o formulário mantendo este elemento vazio, o motor de renderização do navegador interceptará nativamente o gatilho de submissão, impedirá o envio dos dados, aplicará o foco no campo faltante e exibirá uma caixa de mensagem (tooltip) contextualizada informando a obrigatoriedade (MDN WEB DOCS, 2023).

3.2 Atributo pattern (Expressões Regulares)

O atributo pattern mapeia o campo de entrada para uma Expressão Regular (Regex). Esse mecanismo impõe uma máscara matemática e sintática estrita sobre os caracteres digitados. O navegador invalida e bloqueia qualquer dado cuja string total não coincida com a expressão regular estipulada. No cenário nacional brasileiro, para a validação semântica de um telefone celular estruturado com o código de área DDD, utiliza-se a expressão regular `pattern="[0-9]{2} [0-9]{5}-[0-9]{4}"`, forçando o formato "XX XXXXX-XXXX" (W3C, 2021).

3.3 Seletores de tamanho minlength e maxlength

Esses seletores atuam no controle restritivo do comprimento quantitativo das strings manipuladas em campos de texto e áreas de texto estendidas (<textarea>). O atributo minlength estabelece uma quantidade mínima de caracteres (por exemplo, a exigência de ao menos 15 caracteres para uma mensagem de Ouvidoria, sob pena de bloqueio nativo do envio). Já o atributo maxlength estipula uma trava física intransponível na interface; ao atingir o limite numérico superior definido, o navegador bloqueia a inserção de novos caracteres pelo teclado do usuário (MDN WEB DOCS, 2023).

4 Estados Visuais (CSS) e Pseudo-classes :valid and :invalid

O comportamento visual e o design responsivo das folhas de estilo (CSS) sincronizam-se dinamicamente com os estados internos de validação gerados pelo navegador web. As pseudo-classes estruturais permitem estilizar os elementos com base na conformidade dos dados inseridos (SILVA, 2014).

4.1 Pseudo-classes :valid and :invalid

A pseudo-classe :valid é injetada automaticamente pelo motor CSS assim que as restrições e regras estipuladas no campo HTML5 são plenamente satisfeitas. Por outro lado, a pseudo-classe :invalid passa a governar a estilização do componente no momento exato em que há uma quebra de regra (como um e-mail sem formatação adequada ou campo obrigatório em branco).

4.2 Estratégias Avançadas de Usabilidade e Estilização

De acordo com os padrões de design de usabilidade modernos, a aplicação direta da pseudo-classe :invalid sobre elementos globais gera uma falha grave de UX (anti-padrão de interface), dado que o formulário seria renderizado completamente vermelho e com alertas visuais de erro antes mesmo do usuário iniciar a interação de digitação. Para solucionar este comportamento inadequado, a engenharia de estilos determina a associação dos seletores de validação ao seletor de foco dinâmico (:focus), aplicando as cores reativas exclusivamente durante o preenchimento em tempo real (SILVA, 2014), conforme exemplificado abaixo:

CSS

```
/* Ativação visual sob conformidade dos dados durante a interação*/
input:focus:valid,
textarea:focus:valid,
select:focus:valid {
    border: 2px solid #2ecc71;
    background-color: #f4fbf7;
    outline: none;
    box-shadow: 0 0 5px rgba(46, 204, 113, 0.3);}

/* Ativação visual sob inconformidade ou erro durante a interação*/
input:focus:invalid,
textarea:focus:invalid {
    border: 2px solid #e74c3c;
    background-color: #fdf5f5;
    outline: none;
    box-shadow: 0 0 5px rgba(231, 76, 60, 0.3);}
```

5 Conclusão

Conclui-se que a estruturação de um portal interativo exige a adoção indissociável de mecanismos complementares de validação. Enquanto os recursos client-side do HTML5 e intercepções lógicas via JavaScript (como a implementação de exibições prévias e manipulação com `URLSearchParams` exigidas nesta Atividade Prática 07) elevam a usabilidade do usuário e provêm uma interface ágil, as defesas rigorosas implementadas no back-end via server-side blindam a aplicação contra ameaças externas. A convergência mútua de ambas as frentes garante um sistema seguro, estável e amigável.

Referências

FLANAGAN, David. **JavaScript: O Guia Definitivo**. 6. ed. Porto Alegre: Bookman, 2012.

MDN WEB DOCS. **Constraint validation**: Client-side form validation. Mozilla Developer Network, 2023. Disponível em: Biblioteca de Documentação Web da Mozilla. Acesso em: 17 jul. 2026.

SILVA, Maurício Samy. **HTML5: A evolução do Web Design**. 2. ed. São Paulo: Novatec, 2014.

W3C. World Wide Web Consortium. **HTML5.3 W3C Recommendation**: Forms and input validation elements. W3C, 2021. Disponível em: Consórcio Web Internacional. Acesso em: 17 jul. 2026.